AtomQuest

Performance Goal Tracking Portal

A comprehensive performance management system for organizations following the Indian Financial Year (April–March) cycle. Enabling employees to set goals, log quarterly check-ins, and track progress with AI-powered assistance.

Version: 1.0.0

Date: May 2026

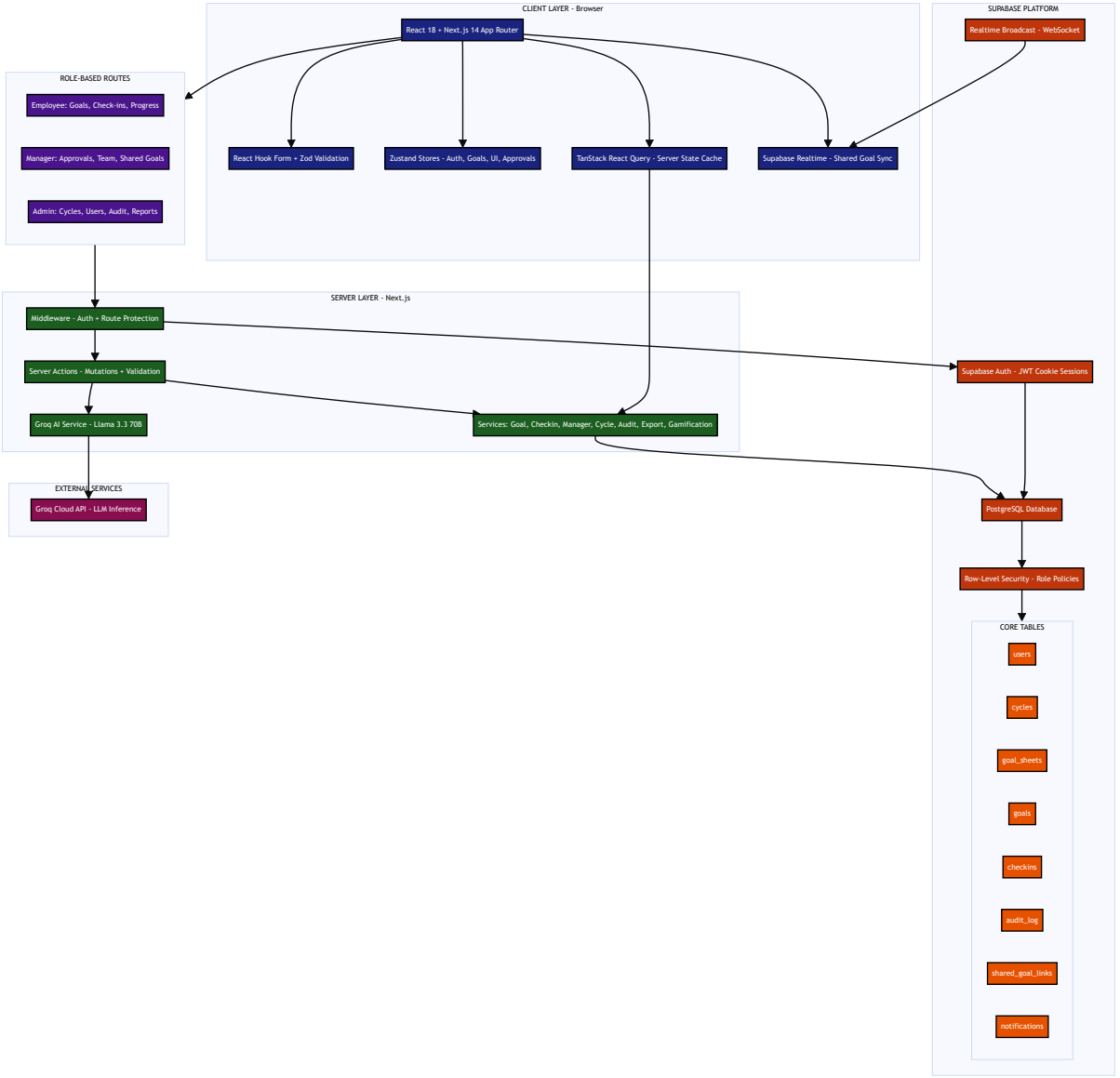
Stack: Next.js 14 | TypeScript | Supabase | Groq AI

Table of Contents

1. System Architecture
2. Tech Stack
3. Features Overview
4. Role-Based Access Control
5. Database Schema
6. Demo Credentials
7. Project Structure
8. Setup & Installation
9. Key Design Decisions

1. System Architecture

The application follows a modern full-stack architecture with clear separation between client, server, and data layers.



2. Tech Stack

Layer	Technology
Framework	Next.js 14.2 (App Router, Server-Side Rendering)
Language	TypeScript 5.7
UI Library	React 18.3, Tailwind CSS 3.4, Radix UI, Lucide Icons
Forms & Validation	React Hook Form 7.x + Zod 3.x
Client State	Zustand 5.x (auth, UI, goal drafts, approvals)
Server State	TanStack React Query 5.x (caching, invalidation)
Database	PostgreSQL via Supabase (triggers, constraints, pg_trgm)
Authentication	Supabase Auth (JWT cookies, SSR-safe sessions)
Real-time	Supabase Realtime (WebSocket broadcast channels)
AI / LLM	Groq Cloud API (Llama 3.3 70B Versatile)
Export	XLSX and CSV report generation
Toast Notifications	Sonner

3. Features Overview

Employee Portal

- Create and manage up to 8 weighted goals per performance cycle
- Log quarterly check-ins with actual progress values (Q1–Q4)
- AI-powered goal writing assistant (generates SMART goals from intent)
- Auto-computed progress scores (0–200%) based on UoM type
- Visual progress dashboard with completion indicators
- In-app notifications for approvals, returns, and deadlines
- Goal sheet submission workflow (draft → submitted → approved/returned)

Manager Portal

- Review and approve/return employee goal sheets with feedback
- Monitor team progress across all quarters
- Create shared/cascaded goals that sync across team members
- View team check-in history and completion rates
- Team performance reports and analytics

Admin Portal

- Manage performance cycles (Indian FY: April–March)
- User management with role assignment (employee/manager/admin)
- Append-only audit log for full compliance tracking
- Unlock locked goal sheets with mandatory reason (audit trail)
- Organization-wide reports with Excel/CSV export
- Escalation management for overdue submissions
- Department and gamification leaderboards

4. Role-Based Access Control

Authorization is enforced at **three independent layers** for defense-in-depth:

Layer	Mechanism	Description
1. Middleware	Next.js Route Guards	Redirects unauthorized users before page loads
2. Database	PostgreSQL RLS Policies	Row-level filtering ensures users only see permitted data
3. Server Actions	Role Validation	Every mutation verifies role before executing

Route Access Matrix

Role	Accessible Routes	Key Permissions
Employee	/home, /goal-sheet, /checkins, /progress, /profile, /notifications	Own goals only, submit goal sheets, log check-ins
Manager	/team, /approvals, /manager-checkins, /shared-goals, /team-reports	View direct reports, approve/return sheets, create shared goals
Admin	/dashboard, /cycles, /users, /audit-log, /reports, /unlock, /escalations, /settings	Full system control, cycle management, audit access

5. Database Schema

Table	Purpose	Key Constraints
`users`	Portal users (1:1 with Supabase Auth)	role enum, manager_id FK, department
`cycles`	Performance cycles with quarterly	Only one active cycle at a time

Table	Purpose	Key Constraints
	gates	
`goal_sheets`	Employee goal container per cycle	Status workflow: draft→submitted→approved/returned
`goals`	Individual goals (max 8 per sheet)	Min 10% weightage, total = 100%, trigger-enforced
`checkins`	Quarterly progress entries	Auto-computed progress_score (0–200%)
`shared_goal_links`	Junction for cascaded KPI sync	source_goal_id, target_goal_id, is_primary_owner
`audit_log`	Immutable compliance ledger	No UPDATE/DELETE policies (append-only)
`notifications`	In-app event notifications	User-scoped, action links

UoM (Unit of Measure) Types

Type	Logic	Example
`numeric_min`	Higher actual = better (actual/target × 100)	Revenue target: ₹10L
`numeric_max`	Lower actual = better (target/actual × 100)	Bug count: target 5
`timeline`	Date-based milestone completion	Launch by June 30
`zero`	Binary (done or not done)	Certification obtained

6. Demo Credentials

Use the following credentials to access the application with different roles:

EMPLOYEE

Employee Account

Email: `omkar1234@gmail.com`

Password: `123456789`

Access: Goal sheet creation, quarterly check-ins, progress tracking, AI goal assistant, notifications

MANAGER

Manager Account

Email: `ankita@gmail.com`

Password: `123456789`

Access: Team overview, goal sheet approvals, shared goals, team check-ins, team reports

ADMIN

Admin Account

Email: `admin@atomquest.dev`

Password: `Admin@12345`

Access: Full system control — cycles, users, audit log, unlock, reports, escalations, settings

7. Project Structure

```
src/ ├── app/ # Next.js App Router │ ├── (admin)/ # Admin routes (dashboard, cycles, users, audit-log, reports, unlock, escalations, settings) │ ├── (auth)/ # Authentication (login, signup) │ ├── (employee)/ # Employee routes (home, goal-sheet, checkins, progress, profile, notifications) │ ├── (manager)/ # Manager routes (team, approvals, manager-checkins, shared-goals, team-reports) │ └── actions/ # Server Actions (admin, ai, auth, checkins, goals, shared-goals) ├── components/ # React components organized by domain │ ├── admin/ # Admin-specific components │ ├── ai/ # AI goal assistant UI │ ├── checkins/ # Check-in forms and views │ ├── goals/ # Goal cards, forms, sheets │ ├── manager/ # Manager-specific components │ ├── shared/ # Shared/common components │ └── ui/ # Radix-based design primitives ├── hooks/ # Custom React hooks (auth, goals, checkins, realtime, team) ├── lib/ # Core utilities │ ├── ai/ # Groq AI service integration │ ├── auth/ # Session management helpers │ ├── supabase/ # Client/server Supabase instances │ └── utils/ # General utilities ├── providers/ # React context providers (Auth, Query, Theme) ├── services/ # Data access layer (8 service classes) ├── store/ # Zustand state stores ├── types/ # TypeScript type definitions └── validations/ # Zod validation schemas supabase/ └── migrations/ # Ordered SQL migrations (001–005)
```

8. Setup & Installation

Prerequisites

- Node.js 18+ installed
- A Supabase project (with Auth enabled)
- Groq API key (for AI goal assistant)

Installation Steps

```
# 1. Clone the repository git clone https://github.com/your-org/atomquest.git cd atomquest # 2. Install dependencies npm install # 3. Configure environment variables cp .env.example .env.local # Edit .env.local with your Supabase and Groq credentials # 4. Run database migrations npm run db:migrate:seed # 5. Start development server npm run dev
```

Environment Variables

Variable	Description
`NEXT_PUBLIC_SUPABASE_URL`	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase anonymous/public key
`SUPABASE_SERVICE_ROLE_KEY`	Supabase service role key (server-only)
`GROQ_API_KEY`	Groq Cloud API key for LLM

Available Scripts

Command	Description
`npm run dev`	Start development server (localhost:3000)
`npm run build`	Production build
`npm run start`	Start production server
`npm run lint`	Run ESLint
`npm run type-check`	TypeScript type validation
`npm run db:migrate`	Run database migrations
`npm run db:migrate:seed`	Run migrations + seed data

9. Key Design Decisions

Decision	Rationale
Max 8 goals per sheet	Keeps employees focused; enforced by DB trigger
Weightage: min 10%, total 100%	Ensures meaningful distribution; validated at all layers
Progress score 0–200%	Allows exceeding targets; auto-computed by Postgres trigger
Append-only audit log	Compliance requirement; no UPDATE/DELETE RLS policies
Real-time shared goals	Primary owner's check-in broadcasts to linked goals via WebSocket
AI server-side only	Groq API key never exposed to client; all calls through Server Actions
Three-layer authorization	Defense-in-depth: middleware + RLS + server action validation
Indian FY cycle (Apr–Mar)	Matches target organization's fiscal year structure
React Query + Zustand	Separates server state (cacheable) from client state (ephemeral)
Supabase SSR auth	Cookie-based JWT avoids hydration issues; middleware refreshes tokens